PHASE 7 – PROJECT DOCUMENTATION

AI FAQ Assistant API – AI-Powered FAQ and Customer Support Automation

---

1. Introduction

The **AI FAQ Assistant API** is a RESTful backend application developed to provide FAQ management and AI-powered customer support functionality.

The project combines a Node.js and Express.js backend with MongoDB for data storage and Google Gemini for AI-based answer and FAQ generation.

The application provides authentication, FAQ management, keyword-based FAQ search, AI services, validation, authorization, and centralized error handling.

This document provides a simplified overview of the complete project, including its purpose, requirements, design, implementation, testing, and outcome.

---

2. Project Overview

The main purpose of the project is to provide a backend API through which users can manage frequently asked questions and use AI services to generate useful responses.

The system provides two major areas of functionality:

FAQ Management

Users can:

- Create FAQs.
- View FAQs.
- View an individual FAQ.
- Update their FAQs.
- Delete their FAQs.
- Search FAQs using keywords.

AI Assistance

The system provides:

- AI-generated answers for questions.
- AI-generated FAQ content based on a topic.

The application uses authentication to protect user-specific operations.

---

3. Problem Statement

Traditional FAQ systems generally require users or administrators to manually create and maintain FAQ content.

Finding relevant information can also become difficult when the number of FAQs increases. In addition, manually preparing answers for every customer question can require additional effort.

The project addresses these problems by providing a centralized API for FAQ management together with AI-powered response generation.

The system allows users to manage FAQ information while using an AI service to generate answers and FAQ content when required.

---

4. Project Objectives

The main objectives of the project are:

1. To develop a RESTful backend API for FAQ management.
2. To provide secure user registration and login.
3. To implement JWT-based authentication.
4. To store FAQ information using MongoDB.
5. To provide FAQ CRUD operations.
6. To provide keyword-based FAQ searching.
7. To integrate Google Gemini for AI-generated answers.
8. To generate FAQ content using AI.
9. To validate API requests.
10. To provide centralized error handling.
11. To test the API using Postman and automated testing.

---

5. Requirements and Proposed System

5.1 Functional Requirements

The system supports the following major functions:

Function	Description
User Registration	Creates a new user account
User Login	Authenticates an existing user
User Profile	Provides authenticated user information
FAQ Creation	Creates a new FAQ
FAQ Retrieval	Retrieves FAQ information
FAQ Update	Updates an FAQ created by the user
FAQ Deletion	Deletes an FAQ created by the user
FAQ Search	Searches FAQs using keywords
AI Answer	Generates an AI-based answer
AI FAQ Generation	Generates FAQ content using AI
Error Handling	Handles application and database errors

5.2 Proposed System

The proposed system provides a centralized backend API that combines FAQ management with AI assistance.

Users authenticate through the API and can perform authorized FAQ operations. The AI services communicate with the configured Google Gemini service to generate responses.

---

6. Technologies Used

The project was developed using the following technologies:

Technology	Purpose
Node.js	Backend runtime
Express.js	REST API framework
JavaScript	Application programming
MongoDB	Database
Mongoose	MongoDB object modeling

Technology	Purpose
JWT	Authentication
bcrypt	Password hashing
Google Gemini API	AI functionality
Postman	API testing
npm	Dependency management

The application follows an MVC-oriented backend organization.

---

7. System Design and Architecture

The application is organized into different layers so that each part of the backend has a specific responsibility.

The general architecture is:

Client / Postman

↓

Express.js API Routes

↓

Middleware

↓

Controllers

↓

Services / Models

↓

MongoDB / Google Gemini

Main Components

Routes handle API endpoints.

Middleware handles authentication, validation, and errors.

Controllers process API requests and responses.

Models define the MongoDB data structures.

Services handle the Gemini AI integration.

Database Configuration manages the MongoDB connection.

The main database models implemented in the project are:

- User
- FAQ

---

8. Project Implementation

The backend is organized into the following major folders:

src/

```
├── config/
│   └── db.js
├── controllers/
│   ├── aiController.js
│   ├── authController.js
│   └── faqController.js
├── middleware/
│   ├── authMiddleware.js
│   ├── errorMiddleware.js
│   └── validationMiddleware.js
├── models/
│   ├── FAQ.js
│   └── User.js
├── routes/
│   ├── aiRoutes.js
│   ├── authRoutes.js
│   └── faqRoutes.js
├── services/
│   └── geminiService.js
├── utils/
```

│ └── helpers.js

├── app.js

└── server.js

Additional project files include:

package.json

package-lock.json

README.md

test_api.js

AI_FAQ_Assistant_API.postman_collection.json

.env

The implementation separates authentication, FAQ management, AI processing, database operations, routing, and middleware.

---

9. Main Features and API Modules

9.1 Authentication Module

The authentication module provides:

- User registration.
- Duplicate email prevention.
- Password hashing using bcrypt.
- User login.
- JWT generation.
- Protected profile access.

Main endpoints:

POST /api/auth/register

POST /api/auth/login

GET /api/auth/profile

9.2 FAQ Management Module

The FAQ module provides complete CRUD functionality.

Main endpoints:

POST /api/faqs

GET /api/faqs

GET /api/faqs/:id

PUT /api/faqs/:id

DELETE /api/faqs/:id

FAQ categories supported by the implementation are:

- Technology
- Education
- Health
- Banking
- General

FAQ modification and deletion include creator verification.

9.3 FAQ Search Module

The application provides keyword-based search through:

GET /api/faqs/search?q=query_string

The search functionality checks relevant FAQ fields including the question, answer, and category.

9.4 AI Module

The AI functionality is implemented through the Gemini service.

AI Answer

POST /api/ai/answer

This endpoint accepts a question and returns an AI-generated response.

AI FAQ Generation

POST /api/ai/generate-faq

This endpoint accepts a topic and generates FAQ content using the configured AI service.

---

10. Security and Error Handling

Security is an important part of the project.

JWT Authentication

JWT is used to authenticate users when accessing protected endpoints.

Password Protection

Passwords are hashed using bcrypt before being stored.

Creator Authorization

FAQ update and delete operations verify the creator of the FAQ before allowing the operation.

Validation

Validation middleware is used to check incoming request data.

Error Handling

The application contains centralized error handling for common situations including:

- Mongoose validation errors.
- Invalid MongoDB object IDs.
- Duplicate database keys.
- Authentication failures.
- AI service errors.

---

11. Testing and Results

The project was tested using **Postman** and the automated test_api.js file.

The major testing areas included:

Test Area	**Result**
User Registration	Tested
Duplicate Registration	Tested
User Login	Tested
JWT Authentication	Tested

Test Area	Result
User Profile	Tested
FAQ Creation	Tested
FAQ Retrieval	Tested
FAQ Update	Tested
FAQ Deletion	Tested
FAQ Search	Tested
Invalid FAQ ID	Tested
AI Answer	Tested
AI FAQ Generation	Tested
Error Handling	Tested

The Postman collection provides individual requests for authentication, FAQ management, search, and AI services.

The automated testing file provides a repeatable API testing sequence covering the major application functions.

---

12. Advantages and Limitations

12.1 Advantages

- RESTful API architecture.
- Secure JWT-based authentication.
- Password hashing using bcrypt.
- MongoDB-based data storage.
- FAQ CRUD functionality.
- Keyword-based FAQ search.
- Google Gemini AI integration.
- Structured project organization.
- Postman testing support.

- Automated API testing support.
- Centralized error handling.

13. Future Enhancements

The project can be further enhanced with additional functionality such as:

1. A dedicated web-based frontend.
2. Advanced semantic or vector-based FAQ search.
3. More detailed user and administration features.
4. FAQ analytics and usage statistics.
5. Improved AI response customization.
6. Additional API monitoring and logging.
7. Deployment to a cloud environment.
8. More extensive automated test coverage.

These enhancements can extend the project from a backend API into a more complete AI-powered customer support platform.

---

14. Conclusion

The **AI FAQ Assistant API** successfully combines FAQ management and AI-powered assistance within a RESTful backend application.

The project implements user authentication, JWT authorization, FAQ CRUD operations, keyword-based search, AI answer generation, AI FAQ generation, validation, and centralized error handling.

The use of **Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, and Google Gemini** provides the technical foundation for the application.

Testing through Postman and the automated API testing file provides evidence of the major implemented functions.

Overall, the completed project demonstrates the development of a structured backend API capable of managing FAQ information and providing AI-assisted customer support functionality.